01 Async Encode Was an Ownership Redesign

Async encode was an ownership redesign.

SYNC

```rust
encode(item, &mut EncodeBuf)
```

borrow caller buffer

return after body bytes exist

Pending can happen

ASYNC

```rust
encode(item, EncodeBuffer)
  -> T::Encode
```

move buffer into future

Ready returns EncodeBuffer

Tonic stores

one owned `T::Encode` per in-flight message

Prost owns

exact field / tag / length / payload resume state inside that future

02 Sync Encode: Complete Body, Then Frame

Sync Tonic frames only after body bytes are complete and the `&mut EncodeBuf` borrow is gone.

Completed-buffer contract

message → borrow `&mut EncodeBuf` → write full body → return complete bytes → Tonic frames

Code evidence

```rust
fn encode(&mut self, item, dst: &mut EncodeBuf) -> Result<()>
```

The `&mut` borrow ends when the call returns.

After return

- size-check
- optional compression
- flags + length
- yield frame

03 Borrowed Async Encode Cannot Be Stored

Tempting but wrong

```rust
async fn encode(
  &mut self,
  item,
  dst: &mut EncodeBuf,
) -> Result<()>
```

What Tonic must store

`in_flight: Option<T::Encode>`

one future · per message · `Send + 'static`

Failure at `Pending`

caller stack owns `&mut` borrows → future returns `Pending` → future still borrows `dst` / `self` → cannot store as `T::Encode: Send + 'static`

Correct storage unit: owned future + owned `EncodeBuffer`.

04 The New Tonic Boundary

encode moves the buffer into a future; Ready returns it for framing.

Encoder API

```rust
type Encode: Future<Output = Result<EncodeBuffer, Self::Error>>
  + Send + 'static

fn encode(self: Pin<&mut Self>, item, dst: EncodeBuffer)
  -> Result<Self::Encode, Self::Error>
```

Body driver slot

```rust
#[pin]
in_flight: Option<T::Encode>
in_flight_offset: Option<usize>
```

stored future, not borrowed buffer

Per-message lifecycle

start: move item + buffer in → poll: Pending keeps future stored → ready: buffer returns

05 Driver Lifecycle: Reserve, Store, Poll, Finish

One message through `EncodedBytes`

1. reserve 5-byte header gap → 2. move active `BytesMut` into `EncodeBuffer` → 3. store `in_flight = Some(T::Encode)` → 4. poll `Pending` / `Ready` → 5. finish compression + header

reserve

```
buf.reserve(HEADER_SIZE)
```

start

```
encoder.encode(item, dst)
```

poll

```
ready!(encode.poll(cx))
```

finish

```
finish_encode_buffer(...)
```

Core point Frame finalization stays in Tonic: compression, max-size check, flags, and gRPC length are written after `Ready`.

06 What Lives Inside `T::Encode`

`DsaAsyncProstEncode<T>`

the object Tonic stores

state
`PollEncodeState` + payload state

item
`Option<T>`

sink
`DsaProstEncodeSink` + buffer

options
length-delimited mode

stage
`Start` / `Body` / `Done`

payload cleanup
descriptor + completion drop path

Pending: keep all compartments → **Ready: `sink.into_inner()` returns `EncodeBuffer`**

Drop order is part of the safety contract: pending payload state may reference item bytes and destination storage.

07 Prost Records the Byte Resume Point

Tonic polls one future; Prost remembers the byte-position checkpoint inside it.

Tonic body driver coarse poll slot

stores and polls one `T::Encode`

↓

owned `T::Encode` future per in-flight message

owns message + buffer + offload state

`PollEncodeState` nested inside the future

byte-level continuation

`PollEncodeFrame` + payload state

- field
- index
- phase
- payload state

CPU writes keys, lengths, and scalars immediately.

Payload copy may return `Pending`.

08 One Suspended Length-Delimited Field

Checkpoint after payload copy returns `Pending`

field N selected
cursor chooses this field

→

tag emitted
do not write again

→

length emitted
do not write again

→

payload copy `Pending`
copy stopped mid-payload

→

next poll resumes payload only
legal continuation

wrong field: skip/repeat

wrong phase: duplicate tag

wrong phase: duplicate length

wrong offset: corrupt payload

The checkpoint is protocol state: tag and length stay written; only the unfinished payload copy continues.

Offload Makes Ownership Physical

Pending offload means raw addresses can outlive the encode call.

owned `T::Encode` holds the physical state

DSA/IAX supplies the pending copy

- source bytes: read address stays valid
- `EncodeBuffer` storage: write address stays stable
- payload state: descriptor + completion stay pinned

If ownership is split, a Rust lifetime bug becomes hardware memory corruption.

source valid

payload backing memory stays allocated while hardware may read it

destination stable

`EncodeBuffer` storage does not move while hardware may write it

payload cleanup drained

payload state's descriptor and completion are owned until Ready or drop cleanup

10 Reusable Rule

If an operation may be stored and polled after the call returns, anything needed after suspension must be owned by that operation.

Tonic owns frame boundary

header reservation, compression, size check, flags, length

→

`T::Encode` owns suspended work

message, `EncodeBuffer`, offload state, completion, cancel/drop cleanup

→

Prost owns wire checkpoint

field, index, phase, and payload-copy continuation

Measurement is downstream

Architecture creates the chance to remove one staged CPU payload copy. Benchmarks decide whether offload submission, wakeup, and resume bookkeeping are smaller.

Async encode = owned resumable state + exact protocol resume points.